Scalable Data Processing and Analytics using Apache Spark

Mining of Massive Datasets (23CSE355) — Assignment 2

Wikimedia editing activity: 10,987,077 revisions through RDDs, DataFrames, Spark SQL and Structured Streaming

Team 22 Project Advisor: Dr Niharika Panda

Sl. No.	Reg. No.	Name of the student
1	BL.EN.U4CSE23206	Aman Agarwal
2	BL.EN.U4CSE23209	Atharva Wasnik
3	BL.EN.U4CSE23245	Pulapa Yaswanth Kumar

Assignment 2: Objectives and Scope

Build a scalable PySpark pipeline that processes, transforms, queries and streams a real-world dataset, exercising all four Spark APIs.

Part	Requirement	Where it is implemented
A	Spark environment: components, driver, cluster manager, executors	src/config.py, slides 3–5
B	RDD creation, map, filter, flatMap, reduceByKey	src/pb_rdd.py
C	DataFrames: schema, cleaning, groupBy, aggregation, Spark SQL	src/p03_prepare.py, src/pc_sql.py
D	Workflow: Input → RDD → DataFrame → Analytical Output	src/run_all.py, slide 6
E	Structured Streaming with per-batch aggregation	src/pe_streaming.py
F	Technical report	reports/Assignment2_Spark_Report.docx (15 pages)

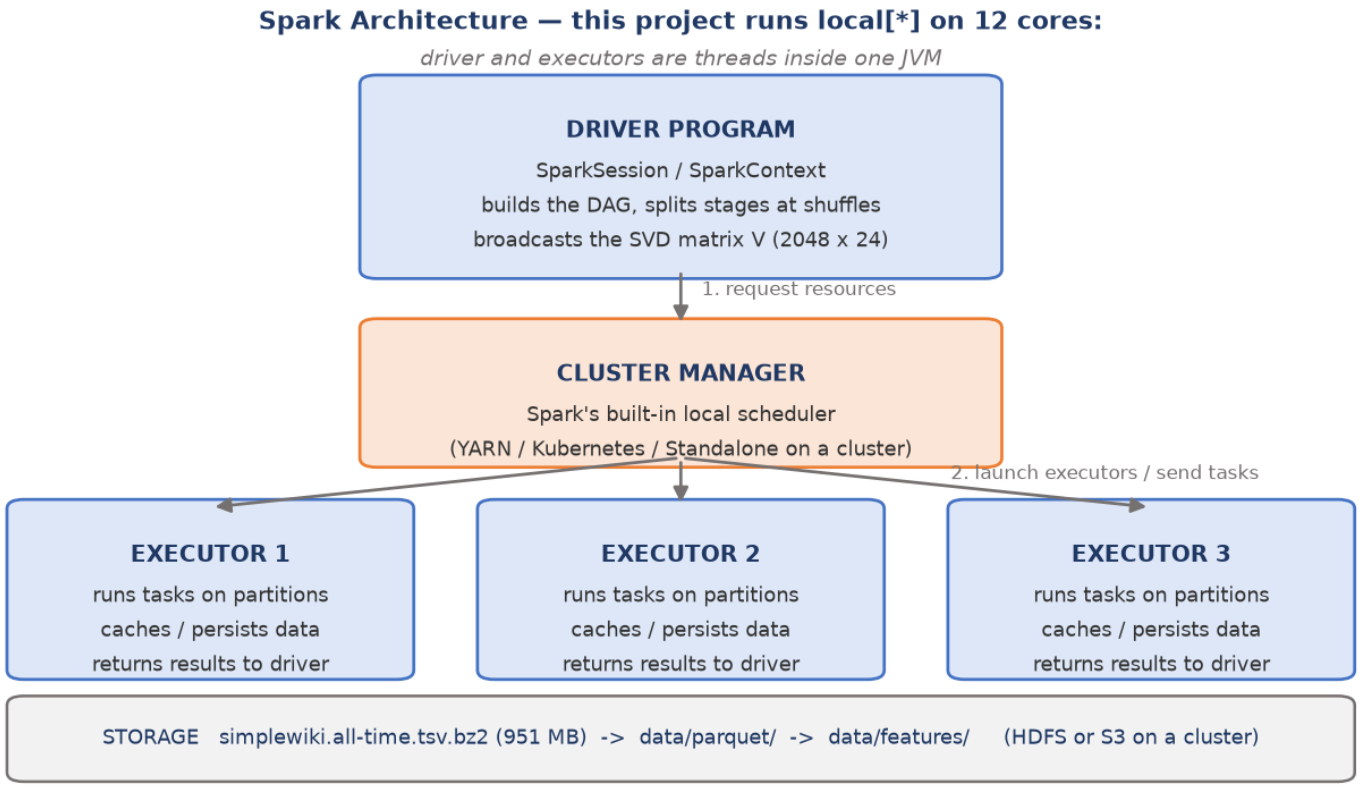
Dataset: Wikimedia MediaWiki History dump, 951 MB bz2, 15,201,370 rows — well above the 10k streaming minimum.

Dataset Description

Item	Details
Source	Wikimedia mediawiki_history dumps, snapshot 2026-10 (CC0)
Wiki / period	simplewiki, complete all-time history (2001–2026)
Download	951 MB bz2 TSV, 78 tab-separated columns, no header
Raw rows	15,201,370 (revision 11,023,987, page 2,231,160, user 1,946,223)
Usable after cleaning	10,987,077 (0.33% removed)
Columns retained	31 of 78
Labels	bot 23.7% · reverted 14.1%
Streaming source	replay of 200,000 records + live EventStreams (SSE)

AMRITA VISHWA VIDYAPEETHAM

Part A: Spark Architecture



Driver builds the DAG and splits stages at shuffles; the cluster manager allocates executors, which run one task per partition. Under local[*] all three are threads in one JVM.

AMRITA VISHWA VIDYAPEETHAM

Part A: Environment and Execution Model

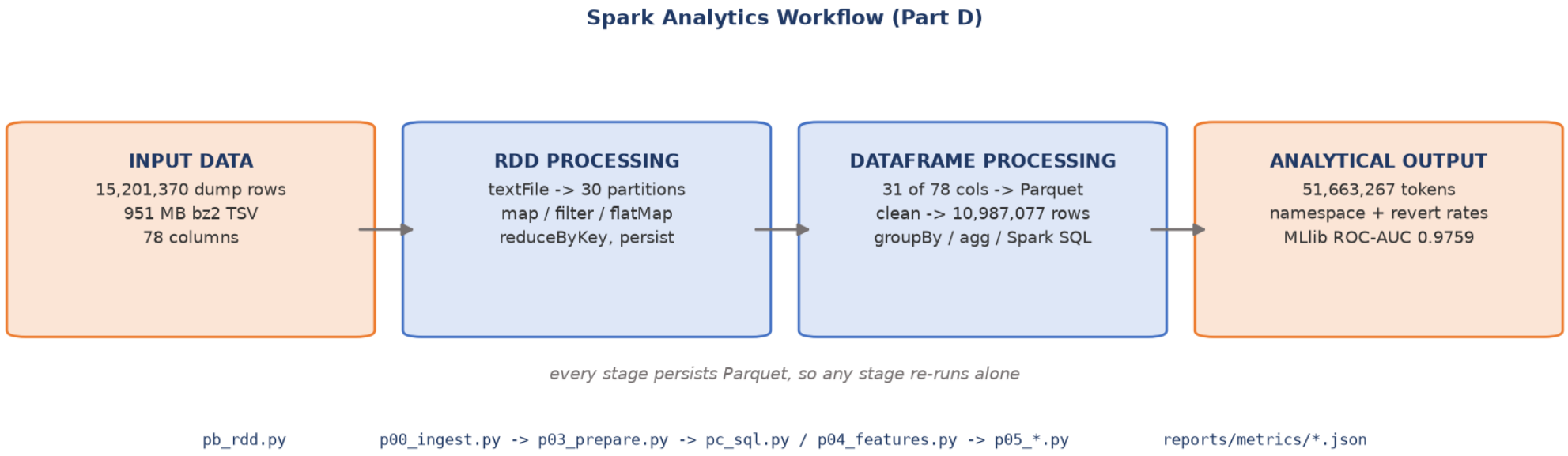
```
src/config.py
SPARK_CONF = {
  "spark.driver.memory": "6g",
  "spark.sql.shuffle.partitions": "48",   # ~4x cores
  "spark.sql.adaptive.enabled": "true",
  "spark.sql.adaptive.skewJoin.enabled": "true",
  "spark.serializer": "...KryoSerializer",
}
MASTER = "local[*]"
```

Concept	In this project
Job	one per action (count, collect, write)
Stage	cut at every shuffle boundary
Task	one per partition — 48 after a shuffle
Narrow op	withColumn, filter — no data movement
Wide op	groupBy, window — full shuffle
Partitions	30 on the raw bz2 read

Lazy evaluation, learned the hard way. Our first ingest called four actions, so Spark replayed the lineage four times and decompressed the 951 MB file **four separate times**. One grouped aggregate now yields the row total, the entity breakdown and the wiki check together: four passes became two.

AMRITA VISHWA VIDYAPEETHAM

Part D: Spark Analytics Workflow



Every stage persists Parquet, so any stage re-runs alone. The RDD and DataFrame branches read the same source, which is what makes the cross-API validation on the next slides possible.

Part B: RDD Creation and Transformations

The RDD phase runs on the **raw compressed dump**, not on Parquet — sc.textFile across 30 partitions, because bzip2 is splittable in Hadoop. A gzip dump would have been one task.

src/pb_rdd.py

```
lines = sc.textFile(str(config.RAW_TSV), minPartitions=24)

parsed = lines.map(parse).filter(lambda r: r is not None)
revs = (parsed.filter(lambda r: r[0] == "revision")
              .map(lambda r: r[1:])
              .persist(StorageLevel.MEMORY_AND_DISK))

# reduceByKey: edits per namespace
ns = (revs.map(lambda r: (r[0], 1))
          .reduceByKey(lambda a, b: a + b)
          .takeOrdered(10, key=lambda kv: -kv[1]))

# flatMap: word count over every edit comment
words = (revs.flatMap(lambda r: TOKEN.split(r[6].lower()))
             .filter(lambda w: len(w) > 2 and w not in STOP)
             .map(lambda w: (w, 1))
             .reduceByKey(lambda a, b: a + b))
```

Operation	Used for
`textFile`	RDD creation, 30 partitions
`map`	parse 78-field TSV line
`filter`	keep revision events
`flatMap`	comment → tokens
`reduceByKey`	namespace, editor, word counts
`takeOrdered`	top-N action
`persist`	share the parse across 5 jobs

Projecting each row to a compact tuple before persisting: full 78-field lists for 11 M rows would not fit in memory.

Part B: Actions, Jobs and Caching

Each action is one Spark job, and each job replays the lineage from source unless an intermediate result is persisted.

Action	Job	Elapsed	Shuffle?
count() — forces the parse	1	61.1 s	no
edits per namespace (reduceByKey)	2	3.1 s	yes
most active editors (reduceByKey)	3	3.8 s	yes
comment word count (flatMap)	4	16.0 s	yes
bot / anonymous aggregates	5	6.9 s	yes

61.1 s, then 3.1 s. The first action pays to decompress and parse 11 M rows; every later action reuses the persisted projection. Without `persist`, jobs 2–5 each repeat that 61-second parse — about five minutes of wasted work.

AMRITA VISHWA VIDYAPEETHAM

Part B: RDD Results

Namespace	Edits
`0`	7,821,994
`3`	894,246
`14`	661,036
`2`	552,165
`10`	402,717
`4`	401,355

Top comment token	Count
`robot`	1,615,801
`adding`	1,428,633
`talk`	1,131,878
`category`	1,069,091
`user`	824,609
`hotcat`	765,561

Actor class	Edits	Revert rate
Human	8,419,962	16.04%
Bot	2,604,025	7.69%
Anonymous	2,044,198	36.68%

51,663,267 comment tokens processed by flatMap. The vocabulary is maintenance vocabulary — `robot`, `adding`, `category`, `hotcat` — consistent with 23.7% of edits being automated.

Anonymous contributors are reverted at 36.7% against 7.7% for bots — a factor of 4.8. Automation is the least contested category on the platform.

Part B: Output Validation — RDD vs DataFrame

The same aggregates are recomputed through the DataFrame API and reconciled against the RDD results. The two must **not** be equal: the DataFrame path drops rows with a null structural key during cleaning, so the RDD figure has to exceed it by exactly that count.

Metric	RDD path	DataFrame path	Delta	Check
revision rows	11,023,987	10,987,077	36,910	PASS
bot edits	2,604,025	2,598,870	5,155	PASS

The 36,910-row gap is exactly the null-key rows Phase 3 discards. An **exact match would be the bug** — it would mean the cleaning step silently did nothing.

Two independent APIs over the same source, reconciled to a known difference. That is output validation, not a printout.

Part C: Schema, Cleaning and the Null Contract

src/p03_prepare.py

```
# deleted pages null the current-state columns but keep *_historical
df = (df.withColumn("page_namespace",
          F.coalesce("page_namespace", "page_namespace_historical"))
        .withColumn("page_title",
          F.coalesce("page_title", "page_title_historical")))

df = df.dropDuplicates(["revision_id"])

# NULL == "true" is NULL in SQL -> becomes NaN downstream
for c in BOOL_COLS:
    df = df.withColumn(c,
        F.coalesce(F.col(c) == F.lit("true"), F.lit(False)))

df = (df.withColumn("label_bot",
          (F.length(F.trim("event_user_is_bot_by")) > 0).cast("int"))
        .withColumn("label_reverted",
          F.col("revision_is_identity_reverted").cast("int")))
```

Step	Rows
Revision rows in	11,023,987
Duplicates removed	0
Null-key rows removed	36,910
Clean rows	**10,987,077**
Bot edits	2,598,870 (23.7%)
Reverted	1,550,915 (14.1%)

14.8% of revisions have a null namespace because the page was deleted. Dropping them would bias the revert label, since deleted pages concentrate vandalism. Recovered from *_historical: loss fell from ~15% to 0.33%.

Part C: Spark SQL Queries

src/pc_sql.py

```
df.createOrReplaceTempView("wikimedia_edits")

# namespace aggregation
spark.sql("""
  SELECT page_namespace, COUNT(*) AS edits,
         ROUND(AVG(label_bot), 4) AS bot_share
  FROM wikimedia_edits
  GROUP BY page_namespace
  ORDER BY edits DESC LIMIT 10""")

# most frequent comment terms
spark.sql("""
  SELECT word, COUNT(*) AS n FROM (
    SELECT explode(split(lower(event_comment),
                   '[^a-z0-9]+')) AS word
    FROM wikimedia_edits)
  WHERE length(word) > 2
  GROUP BY word ORDER BY n DESC LIMIT 15""")
```

ns	Namespace	Edits	Bot %	Revert %
0	Article	7,799,888	23.2	12.8
3	User talk	893,306	18.5	11.2
14	Category	659,831	53.5	3.7
2	User	550,127	27.2	30.0
10	Template	401,997	20.1	10.2
4	Project	401,011	7.3	45.7

Actor segment	Edits	Revert rate
registered human	6,357,719	9.45%
bot	2,598,870	7.70%
anonymous	2,030,488	36.92%

DataFrame API 1.02s vs Spark SQL 0.66s — identical Catalyst plan. The choice is expression, not performance.

Part C: Analytical Findings

Category space is 53.5% automated against a 23.7% corpus average, and has the lowest revert rate of the major namespaces at 3.7%. Category maintenance is largely machine work and largely uncontested.

Anonymous editors are reverted 4.8x more often than bots — 36.92% against 7.70%, with registered humans between them at 9.45%.

Automated editing peaked and receded. Activity peaked in 2012 with 771,344 edits at 45.3% automation. The busiest recent month, 2025-05, runs at only 6.5%.

Most edited article: `Deaths_in_2020` with 8,538 revisions.

Both APIs answer the term-frequency question and agree on the ordering: robot, adding, talk, category.

AMRITA VISHWA VIDYAPEETHAM

Part E: Structured Streaming

src/pe_streaming.py

```
stream = (spark.readStream
          .schema(SCHEMA)                    # no inference on a stream
          .option("maxFilesPerTrigger", 1) # one burst per micro-batch
          .json(str(STREAM_IN)))

q1 = (stream.writeStream
      .foreachBatch(per_batch)
      .option("checkpointLocation", str(CKPT / "q1"))
      .trigger(processingTime="2 seconds").start())

# event-time windows with a watermark for late data
windowed = (stream
    .withWatermark("event_timestamp", "365 days")
    .groupBy(F.window("event_timestamp", "365 days"))
    .agg(F.count("*").alias("edits"),
         F.avg("label_bot").alias("bot_share")))
```

Property	Value
Source	file source (JSON)
Trigger	processingTime = 2 s
Bursts	20 x 10,000
Micro-batches	20
Records	200,000
Running bot rate	44.11%
Event-time windows	12

A file source was chosen over a socket: a socket source is not fault-tolerant and dies with its producer. The file source checkpoints offsets and is restartable.

Part E: Micro-Batch Output and Interpretation

```
make streaming
[producer] sampled 200,000 edits (44.1% bot)
BATCH  0: rows=10,000 bot=1,945 (19.4%) | running 19.45%
BATCH  5: rows=10,000 bot=4,014 (40.1%) | running 32.39%
BATCH 10: rows=10,000 bot=5,083 (50.8%) | running 38.57%
BATCH 13: rows=10,000 bot=5,489 (54.9%) | running 41.52%
BATCH 19: rows=10,000 bot=6,404 (64.0%) | running 44.11%

event-time windows produced: 12
  2002:    172 edits, bot share  1.7%
  2003:    906 edits, bot share 10.7%
  2004:  2,647 edits, bot share 14.1%
  2005:  9,363 edits, bot share 26.1%
```

The bot rate climbs 19.4% → 64.0%.

That is not noise. The replay is in event-time order, oldest first, so this is the real growth of automated editing on Wikipedia played back in 90 seconds.

The running rate converges on **44.11%**, matching the 44.1% bot share of the sampled replay set measured before streaming began.

Streaming and batch paths agree on the same corpus — the useful cross-check.

Part E: A Partitioning Bug Surfaced by Streaming

Symptom. The first streaming run reported a bot rate of exactly **zero** in every micro-batch.

Cause. Not in the streaming code. The prepared dataset is written with `partitionBy("label_bot")`, so it is laid out on disk as two directories. A `.limit()` over it reads whole partition directories in order, and `label_bot=0` alone holds more rows than the replay needed — so the sample never reached the bot partition.

Fix. Sample across all partitions, then order by event time.

General lesson. `limit()`, `head()` and `take()` are not random samples. On partitioned data they return whatever the first partitions hold — and if the data is partitioned by the column under measurement, the result is perfectly biased with no error raised.

Results and Discussion

- **Scale.** 10,987,077 cleaned revisions processed from a 951 MB compressed dump; 51,663,267 comment tokens counted via flatMap.
- **Revert rate separates actor classes sharply** — anonymous 36.9%, registered human 9.4%, bot 7.7%. Automation is the least contested category on the platform.
- **Automation is concentrated by namespace** — Category is 53.5% automated against a 23.7% average.
- **Streaming reproduces the batch result incrementally** — running bot rate converges on 44.11% across 20 micro-batches.
- **API choice is expression, not performance** — DataFrame 1.02s vs Spark SQL 0.66s on the same question, because both compile to the same Catalyst plan.
- **Caching dominates RDD cost** — the first action took 61.1 s to parse 11 M rows; the next four took 3–16 s each off the persisted projection.

AMRITA VISHWA VIDYAPEETHAM

Scalability Observations

Stage	Parallelisation mechanism
Raw TSV read	bzip2 IS splittable, so the 951 MB file was read by 30 concurrent tasks (gzip would be one)
Column projection	Parquet keeps 31 of 78 columns, so later phases read about a third of the bytes
Feature engineering	row-local withColumn — embarrassingly parallel, no shuffle
Editor aggregation	groupBy + window lag → sort-merge shuffle across 48 partitions; the expensive step
Model training	MLlib broadcasts the model, reduces statistics per partition

Actor skew is the dominant risk at scale. Edits per account follow a heavy power law — `Auntof6` has 335,023 revisions while the median account has one. A naive groupBy sends one enormous partition to a single task, which straggles or OOMs while the other cores idle. Adaptive skew-join handling is enabled; salting the heaviest keys is the next step.

Other limits: small-files problem on write, broadcast ceiling on the SVD matrix if TF-IDF buckets grow, and local[*] only — scalability is argued from observed stage behaviour, not a measured multi-node speed-up.

Reproducing the Results

```
make help
source env.sh             # pins JAVA_HOME and PYSPARK_PYTHON

make smoke                # all phases on 49,478 rows, ~1 min
make rdd                  # Part B: RDD processing          ~90 s
make sql                  # Part C: DataFrame + Spark SQL   ~60 s
make streaming            # Part E: Structured Streaming    ~90 s
make all                  # the full pipeline               ~55 min

make report-a2            # rebuild the 15-page report
make ppt-a2               # rebuild this deck
```

Every phase persists Parquet, so any phase re-runs alone. Both documents are generated from reports/metrics/*.json, so no number in them is typed by hand.

Conclusion

A complete Spark pipeline over 10,987,077 real Wikimedia revisions, exercising all four programming interfaces on one corpus: RDD transformations on the raw compressed dump, DataFrame cleaning and Spark SQL over Parquet, Structured Streaming with watermarked event-time windows, and distributed MLlib.

Three engineering lessons that generalise:

Published schemas may not match shipped data — the docs say 76 columns, the snapshot ships 78. Verifying column order empirically cost less than debugging a misaligned pipeline would have.

Lazy evaluation means every action replays lineage from source. One `persist` turned five re-parses into seconds.

A metric must be read against its own baseline — and a `.limit()` on partitioned data is not a random sample.

Future scope: multi-node cluster with measured speed-up; connect the live EventStreams collector directly to the streaming consumer; extend to larger wikis where actor skew binds.

THANK YOU